

RAPPORT TECHNIQUE DE DÉVELOPPEMENT

Implémentation du Module "Job Board" sous Drupal

Réalisé par

YAHYA EL OMARI

Encadrants

ABDELMAJID BENDRIF

HAMZA BAHLAOUANE

9 mars 2026

Table des matières

Introduction	2
1 Déclaration et Configuration du Module	2
1.1 Fichier d'information : job_board.info.yml	2
1.2 Routage des Requêtes : job_board.routing.yml	2
1.3 Intégration au Menu : job_board.links.menu.yml	3
1.4 Droits d'Accès : job_board.permissions.yml	3
2 Modélisation des Données (Entity API)	3
2.1 Le Contrat d'Interface : JobApplicationInterface	3
2.2 L'Implémentation de Base : ContentEntityBase	4
2.2.1 Définition des Champs (Field API) et Traduction	6
2.2.2 Gestion des Fichiers (target_id) et Fichiers Gérés (Managed Files)	6
3 Couche de Service et Processus Métier	6
3.1 Préservation des Fichiers (setPermanent())	6
3.2 Intégration du MailManager avec hook_mail	6
4 Formulaire Frontal et Bloc	8
4.1 La Classe de Formulaire : JobApplicationForm	8
4.2 Encapsulation via Bloc : JobApplicationBlock	10
5 Contrôleurs, Dashboard et Vues	10
5.1 Affichage du Tableau de Bord : JobBoardAdminController	10
5.2 Traitement des Décisions RH : JobBoardActionController	11
5.3 Gabarit Visuel : job-board-admin-dashboard.html.twig	12
6 Modificateurs Système (Hooks)	13
6.1 Fichier job_board.module	13
7 Fluidité de l'Interface d'Administration (JavaScript)	14
8 Conclusion	15

INTRODUCTION

Note : La genèse de ce module découle directement de nos échanges concernant le groupe Atos lors de notre récente réunion Google Meet. Il a été pensé comme un tremplin d'exploration pour expérimenter et repousser les limites des différents champs avancés mis à disposition par la Form API de Drupal.

Ce rapport technique détaille l'architecture et l'implémentation d'un module Drupal sur mesure nommé "Job Board". Conçu comme un Produit Minimum Viable (MVP), ce module permet la soumission de candidatures en ligne côté public et fournit un tableau de bord d'administration côté backend pour le traitement par les ressources humaines (acceptation ou rejet). Le document expose les principes architecturaux adoptés, en insistant sur l'utilisation de l'API Entity de Drupal, la séparation des préoccupations, et la gestion sécurisée des fichiers soumis.

1 Déclaration et Configuration du Module

1.1 Fichier d'information : job_board.info.yml

L'initialisation de tout module Drupal requiert un fichier de déclaration. Ce manifeste YAML informe le cœur Drupal de l'existence du module, définit ses métadonnées (nom, description, package) et spécifie ses dépendances structurelles, telles que le module **block** requis pour l'affichage de notre formulaire.

```
# Nom affiche du module
name: 'Job Board'
# Type d'extension (module, theme, profile)
type: module
# Courte description visible dans l'administration des extensions
description: 'Système MVP de candidature en ligne.'
# Regroupement dans un dossier spécifique de l'UI
package: Custom
# Compatibilité avec les versions Majeures de Drupal
core_version_requirement: ^10 || ^11
# Liste des modules core ou contribues dont ce module depend pour fonctionner
dependencies:
  - drupal:block
```

1.2 Routage des Requêtes : job_board.routing.yml

Afin d'exposer des pages et des points d'ancrage fonctionnels, les routes HTTP doivent être explicitement déclarées. Ce fichier associe des motifs d'URL (URI) à des classes PHP spécifiques (les Contrôleurs). Il gère également le contrôle d'accès en exigeant la permission **administrer job board** pour sécuriser les routes du back-office.

```
# Identifiant unique de la route pour le tableau de bord
job_board.admin_dashboard:
  path: "/admin/config/content/job_board"
  defaults:
    # Action du controleur a appeler (Classe::Methode)
    _controller: '\Drupal\job_board\Controller\JobBoardAdminController::dashboard'
    # Titre de la page genere
    _title: "Job Board Applications"
  requirements:
    # Droit requis pour interroger cette route (securite d'accès)
    _permission: "administrer job board"

# Route dynamique de type GET pour traiter une acceptation
job_board.action.accept:
  path: "/admin/config/content/job_board/accept/{id}" # {id} = variable passante
  defaults:
```

```

    _controller: '\Drupal\job_board\Controller\JobBoardActionController::accept'
    requirements:
    _permission: "administer job board"

# Route dynamique de type GET pour traiter un refus
job_board.action.reject:
  path: "/admin/config/content/job_board/reject/{id}"
  defaults:
    _controller: '\Drupal\job_board\Controller\JobBoardActionController::reject'
  requirements:
    _permission: "administer job board"

```

1.3 Intégration au Menu : job_board.links.menu.yml

Pour faciliter la navigation des administrateurs, la route du tableau de bord est injectée dans l'arborescence native des menus de Drupal, sous la section existante "Configuration > Contenu".

```

# L'identifiant de ce lien correspond au nom de la route parente
job_board.admin_dashboard:
  title: "Job Board Applications"
  description: "View and manage submitted job applications."
  # Indique a quelle route renvoyer lors du clic
  route_name: job_board.admin_dashboard
  # Placer ce lien en dessous de la section d'administration standard
  parent: system.admin_config_content

```

1.4 Droits d'Accès : job_board.permissions.yml

La sécurité est primordiale dans toute application web. Afin de protéger l'accès au tableau de bord d'administration et de limiter l'attribution de soumission au formulaire, nous devons créer des permissions spécifiques. Ces clés d'autorisation personnalisées sont ensuite associées à des rôles utilisateurs, depuis l'interface native de Drupal.

```

# Permission reservee aux administrateurs (RH)
administer job board:
  title: 'Administer Job Board'
  description: 'Allow users to review and manage submitted job applications.'
  restrict access: true # Avertit Drupal qu'il s'agit d'un droit sensible

# Permission standard allouee aux candidats
submit job application:
  title: 'Submit Job Application'
  description: 'Allow users to view and submit the job application form.'

```

2 Modélisation des Données (Entity API)

L'approche moderne sous Drupal pour stocker des données structurées consiste à utiliser des entités de contenu (Content Entities) au lieu d'interagir directement avec le schéma SQL.

2.1 Le Contrat d'Interface : JobApplicationInterface

Conformément aux principes de la programmation orientée objet, la structure de l'entité est d'abord définie par une interface. L'interface **JobApplicationInterface** hérite de **ContentEntityInterface**, qui est le contrat fondamental défini par le cœur de Drupal pour toute entité de contenu.

L'héritage de **ContentEntityInterface** est impératif : il garantit que notre entité personnalisée disposera de toutes les méthodes de base attendues par les sous-systèmes de Drupal (sauvegarde, chargement, traduction, révision), tout en nous permettant d'y ajouter nos propres accesseurs et mutateurs (getters/setters).

```
<?php
namespace Drupal\job_board;
use Drupal\Core\Entity\ContentEntityInterface;

// Notre contrat d'Entity herite obligatoirement de toutes les contraintes de base de Drupal
interface JobApplicationInterface extends ContentEntityInterface {
    // Lecture / Ecriture des donnees textuelles
    public function getApplicantName(): string;
    public function setApplicantName(string $name): self;

    // Les dates et telephones peuvent etre nullables (?)
    public function getStartDate(): ?string;
    public function setStartDate(string $date): self;
    public function getPhone(): ?string;
    public function setPhone(string $phone): self;

    // Lecture / Ecriture des identifiants (References Cibles) de Fichiers
    public function getCvId(): ?int;
    public function setCvId(int $fid): self;
    public function getCoverLetterId(): ?int;
    public function setCoverLetterId(int $fid): self;

    // Le statut de la candidature (pending, application_accepted, etc.)
    public function getStatus(): string;
    public function setStatus(string $status): self;
}
```

2.2 L'Implémentation de Base : ContentEntityBase

La classe concrète **JobApplication** hérite de **ContentEntityBase**. Cette classe de base abstraite fournie par Drupal implémente la majeure partie de la logique de bas niveau requise par **ContentEntityInterface**. En étendant **ContentEntityBase**, nous délégons à Drupal la gestion complexe du cycle de vie de l'entité, du routage SQL et de l'intégration avec la Field API.

```
<?php
namespace Drupal\job_board\Entity;

use Drupal\Core\Entity\ContentEntityBase;
use Drupal\Core\Entity\EntityTypeInterface;
use Drupal\Core\Field\BaseFieldDefinition;
use Drupal\Core\StringTranslation\TranslatableMarkup;
use Drupal\job_board\JobApplicationInterface;

/**
 * @ContentEntityType(
 *   id = "job_application",
 *   label = @Translation("Job Application"),
 *   base_table = "job_application",
 *   entity_keys = {"id" = "id", "uuid" = "uuid", "label" = "applicant_name"},
 * )
 */
class JobApplication extends ContentEntityBase implements JobApplicationInterface {

    // --- ACCESSEURS ET MUTATEURS DE L'ENTITE ---
    // Les methodes get() et set() de l'API Field gerent l'extraction et l'ecriture.
    // L'operateur null coalescing (??) evite les erreurs si le champ est vide.
    public function getApplicantName(): string { return $this->get('applicant_name')->value ?? ''; }
    public function setApplicantName(string $name): self { $this->set('applicant_name', $name); return $this; }
    public function getStartDate(): ?string { return $this->get('start_date')->value; }
```

```

public function setStartDate(string $date): self { $this->set('start_date', $date); return
    $this; }
public function getPhone(): ?string { return $this->get('phone')->value ?? null; }
public function setPhone(string $phone): self { $this->set('phone', $phone); return $this;
    }

// Pour les champs "Reference d'Entite" (Entity Reference), on utilise 'target_id'
// et non 'value' pour recuperer ou assigner la cle etrangere.
public function getCvId(): ?int { return $this->get('cv')->target_id ?? null; }
public function setCvId(int $fid): self { $this->set('cv', $fid); return $this; }
public function getCoverLetterId(): ?int { return $this->get('cover_letter')->target_id ??
    null; }
public function setCoverLetterId(int $fid): self { $this->set('cover_letter', $fid);
    return $this; }

// Le statut par default est "pending" (en attente)
public function getStatus(): string { return $this->get('status')->value ?? 'pending'; }
public function setStatus(string $status): self { $this->set('status', $status); return
    $this; }

// --- DEFINITION DU SCHEMA DE LA BASE DE DONNEES ---
public static function baseFieldDefinitions(EntityTypeInterface $entity_type) {
    // Chargement des colonnes obligatoires (id, uuid) depuis la classe parente
    $fields = parent::baseFieldDefinitions($entity_type);

    // Colonne VARCHAR(255) pour le nom complet, paramètre obligatoire (NOT NULL)
    $fields['applicant_name'] = BaseFieldDefinition::create('string')
        ->setLabel(new TranslatableMarkup('Name'))
        ->setRequired(TRUE);

    // Colonne Email avec validation native du format e-mail
    $fields['email'] = BaseFieldDefinition::create('email')
        ->setLabel(new TranslatableMarkup('Email'))
        ->setRequired(TRUE);

    // Colonne de caracteres simple pour le telephone (valide plus tard par RegEx)
    $fields['phone'] = BaseFieldDefinition::create('string')
        ->setLabel(new TranslatableMarkup('Phone'))
        ->setRequired(TRUE);

    // Jointure avec la table 'file_managed' : ce champ stockera le File ID (integer)
    $fields['cv'] = BaseFieldDefinition::create('entity_reference')
        ->setLabel(new TranslatableMarkup('CV Upload'))
        ->setSetting('target_type', 'file')
        ->setRequired(TRUE);

    $fields['cover_letter'] = BaseFieldDefinition::create('entity_reference')
        ->setLabel(new TranslatableMarkup('Cover Letter Upload'))
        ->setSetting('target_type', 'file')
        ->setRequired(TRUE);

    // Colonne Date sans l'heure (datetime_type = date)
    $fields['start_date'] = BaseFieldDefinition::create('datetime')
        ->setLabel(new TranslatableMarkup('Start Date'))
        ->setRequired(TRUE)
        ->setSetting('datetime_type', 'date');

    // Colonne VARCHAR(255) representant l'etat RH, s'instanciant avec 'pending'
    $fields['status'] = BaseFieldDefinition::create('string')
        ->setLabel(new TranslatableMarkup('Status'))
        ->setDefaultValue('pending');

    // Capture automatique du timestamp UNIX lors du calcul de $application->save()
    $fields['created'] = BaseFieldDefinition::create('created')
        ->setLabel(new TranslatableMarkup('Created'));

    return $fields;
}
}

```

2.2.1 Définition des Champs (Field API) et Traduction

La méthode statique **baseFieldDefinitions** construit le schéma de la base de données. Il est important de noter l'utilisation de la classe **TranslatableMarkup** au lieu de la fonction globale **t()** habituelle. Dans des contextes statiques, la fonction exécutoire **t()** ne peut pas être correctement résolue par les analyseurs typographiques (IDE) sans le chargement de l'environnement complet de Drupal. Instancier directement l'objet **TranslatableMarkup** garantit la validité stricte du code orienté objet.

2.2.2 Gestion des Fichiers (target_id) et Fichiers Gérés (Managed Files)

Dans Drupal, les fichiers téléchargés (comme les CV en format PDF) ne sont jamais stockés physiquement dans la base de données (BLOB) ni sous forme de simples chemins de chaîne de caractères. Ils sont convertis en entités propres nommées "File Entities" (ou fichiers gérés).

La table **job_application** de la base de données ne contient qu'une colonne d'entier (le **target_id**), agissant comme une clé étrangère qui pointe vers l'identifiant unique (**fid**) du fichier stocké dans la table native **file_managed**. L'utilisation du type de champ **entity_reference** avec le paramètre **target_type** réglé sur **file** instruit Drupal de maintenir cette relation intègre de manière automatisée.

3 Couche de Service et Processus Métier

Afin d'assurer le découplage de l'architecture, une pure classe de service est introduite. La responsabilité des formulaires et des contrôleurs se limite à capturer les entrées HTTP; la validation et le traitement des modèles métiers sont l'apanage de **JobBoardService**.

3.1 Préservation des Fichiers (**setPermanent()**)

Un aspect critique de la logique métier réside dans le cycle de vie des "Managed Files". Lorsque le formulaire définit un champ avec la syntaxe **#type => 'managed_file'**, le comportement par défaut (et forcé) de Drupal est d'enregistrer le fichier avec le statut interne "Temporaire" dans la base de données. Par mesure de sécurité et de nettoyage, le cœur de Drupal exécute un **cron** qui supprime physiquement du serveur tout fichier temporaire âgé de plus de six heures.

Si la soumission d'une candidature ne modifie pas ce statut, les CV téléchargés seront purgés silencieusement. Par conséquent, lors de la persistance de l'entité **JobApplication**, le service extrait d'abord l'identifiant du fichier. Étant donné que la Form API renvoie la valeur du **managed_file** sous forme de tableau (ex. **[0 => fid]**), la fonction native PHP **reset()** est exploitée pour pointer le curseur interne vers le début du tableau et extraire ce premier **fid**. Ensuite, le service charge explicitement les entités **File** liées via cet identifiant et appelle la méthode **setPermanent()**. Cette action modifie l'état du fichier dans la table **file_managed**, empêchant ainsi la purge programmée lors du passage du processus **cron**.

3.2 Intégration du MailManager avec hook_mail

Pour l'envoi de courriels transactionnels, le service délègue la tâche au **MailManager** de Drupal. La méthode **\$this->mailManager->mail('job_board', \$mail_key, ...)** ne contient formellement aucun corps de message. Elle indique uniquement au cœur

d'invoquer le crochet système `hook_mail()` défini ultérieurement dans le fichier `job_board.module`, où la construction textuelle du mail est véritablement opérée selon la `$mail_key` fournie.

```
<?php
namespace Drupal\job_board\Service;

use Drupal\Core\Entity\EntityTypeManagerInterface;
use Drupal\Core\Logger\LoggerChannelFactoryInterface;
use Drupal\Core\Mail\MailManagerInterface;

class JobBoardService {

    // Injection des services du coeur necessaires a notre logique metier
    protected $entityTypeManager;
    protected $loggerFactory;
    protected $mailManager;

    public function __construct(EntityTypeManagerInterface $entity_type_manager,
        LoggerChannelFactoryInterface $logger_factory, MailManagerInterface $mail_manager) {
        $this->entityTypeManager = $entity_type_manager;
        $this->loggerFactory = $logger_factory;
        $this->mailManager = $mail_manager;
    }

    public function processApplication(array $values) {
        try {
            // Acces au depot des entites "JobApplication"
            $storage = $this->entityTypeManager->getStorage('job_application');

            // Le type 'managed_file' de la Form API retourne un tableau d'identifiants (ex:
            // [0 => 123]).
            // Array reset() recupere la premiere valeur (le "fid" - File ID) du tableau.
            $cv_fid = !empty($values['cv']) ? reset($values['cv']) : null;
            $cover_letter_fid = !empty($values['cover_letter']) ? reset($values['cover_letter']) : null;

            // Instanciation en ramenee vive de la nouvelle candidature
            $application = $storage->create([
                'applicant_name' => $values['applicant_name'],
                'email' => $values['email'],
                'phone' => $values['phone'],
                'cv' => $cv_fid, // L'ID du fichier est relie au champ reference
                'cover_letter' => $cover_letter_fid,
                'start_date' => $values['start_date'],
            ]);
            // Execution de la requete SQL INSERT via l'ORM assistie
            $application->save();

            // Modification obligatoire de l'etat des fichiers temporels en permanents
            // afin d'empecher la destruction physique par le Cron de Drupal.
            if ($cv_fid) {
                $cv_file = $this->entityTypeManager->getStorage('file')->load($cv_fid);
                if ($cv_file) { $cv_file->setPermanent(); $cv_file->save(); }
            }
            if ($cover_letter_fid) {
                $cl_file = $this->entityTypeManager->getStorage('file')->load($cover_letter_fid);
                if ($cl_file) { $cl_file->setPermanent(); $cl_file->save(); }
            }

            // Inscription silencieuse d'une entree INFO dans le Watchdog interne (Recent log messages)
            $this->loggerFactory->get('job_board')->info('New application: @name', ['@name' => $values['applicant_name']]);

            // Notification autonome du candidat
            $this->sendNotificationEmail($application);

            return TRUE;
        } catch (\Exception $e) {
            // En cas de defaillance (Erreur SQL par ex.), la transaction avorte silencieusement
        }
    }
}
```



```

        return FALSE;
    }
}

protected function sendNotificationEmail($application) {
    try {
        $applicant_email = $application->get('email')->value;
        // Interception: abandon sans erreur si l'e-mail est inexistant.
        if (empty($applicant_email)) return;

        // Appel du service MailManager de Drupal.
        // Note : Ceci déploie silencieusement la logique de construction textuelle
        // definie dans job_board_mail() situee dans le fichier racine .module.
        $this->mailManager->mail('job_board', 'application_received', $applicant_email, \
            Drupal::currentUser()->getPreferredLangcode(), ['application' => $application],
            NULL, TRUE);
    } catch (\Exception $e) {}
}

public function processAction($id, $mail_key, $action_name) {
    $storage = $this->entityTypeManager->getStorage('job_application');
    // Recuperation asynchrone de l'Entite cible via son identifiant numerique
    $application = $storage->load($id);

    if (!$application) return null; // Avortement preventif

    $to = $application->get('email')->value;
    if ($to) {
        // Injection de theorie: On envoie l'e-mail AVANT de persister le statut RH
        // pour garantir la transactionnalite (ne pas marquer 'accepte' si le reseau Mail
        // coupe).
        $result = $this->mailManager->mail('job_board', $mail_key, $to, \Drupal::
            currentUser()->getPreferredLangcode(), ['application' => $application], NULL,
            TRUE);

        if ($result['result'] === TRUE) {
            // Sauvegarde de l'etat d'acceptation/rejet dans la base de donnees via le
            // setter et save()
            $application->setStatus(strtolower($action_name));
            $application->save();

            // Remontee Audit/Debug
            $this->loggerFactory->get('job_board')->info('Application @id was @action.', [
                '@id' => $id, '@action' => strtolower($action_name)]);
            return true;
        }
    }
    return false;
}
}

```

4 Formulaire Frontal et Bloc

4.1 La Classe de Formulaire : JobApplicationForm

Le module propose une interface utilisateur via la "Form API" de Drupal. Tel qu'explicitée précédemment, la syntaxe `#type => 'managed_file'` génère automatiquement les widgets AJAX requis pour la gestion asynchrone des médias exposés, et astreint ces fichiers à une durée de vie temporaire par défaut. La méthode `validateForm` est surchargée pour vérifier la conformité des numéros de téléphone à l'aide d'expressions régulières (RegEx). La méthode `submitForm` capte les entrées pures et orchestre l'appel au service métier.

```

<?php
namespace Drupal\job_board\Form;

use Drupal\Core\Form\FormBase;

```

```

use Drupal\Core\Form\FormStateInterface;
use Symfony\Component\DependencyInjection\ContainerInterface;
use Drupal\job_board\Service\JobBoardService;

class JobApplicationForm extends FormBase {

    protected $jobBoardService;

    // Injection de la dependance (Inversion of Control) via le conteneur de services
    public function __construct(JobBoardService $job_board_service) { $this->jobBoardService =
        $job_board_service; }
    public static function create(ContainerInterface $container) { return new static(
        $container->get('job_board.service')); }

    // Identifiant unique obligatoire du formulaire au sein de l'ecosysteme Drupal
    public function getFormId() { return 'job_board_application_form'; }

    // Construction dynamique (declarative) des elements du formulaire via le Render Array
    public function buildForm(array $form, FormStateInterface $form_state) {
        $form['start_date'] = ['#type' => 'date', '#title' => $this->t('Available Start Date')]
        ];
        $form['applicant_name'] = ['#type' => 'textfield', '#title' => $this->t('Full Name'),
            '#required' => TRUE];
        $form['email'] = ['#type' => 'email', '#title' => $this->t('Email Address'), '#
            required' => TRUE];
        $form['phone'] = ['#type' => 'tel', '#title' => $this->t('Phone Number (06 or 07)'), '#
            required' => TRUE];

        // Definition des champs de fichiers complexes (managed_file)
        // #upload_validators est un garde-fou natif assurant que le fichier telecharge est
        // bien un .pdf
        $form['cv'] = ['#type' => 'managed_file', '#title' => $this->t('Upload CV (PDF)'), '#
            required' => TRUE, '#upload_validators' => ['FileExtension' => ['extensions' => '
            pdf']]];
        $form['cover_letter'] = ['#type' => 'managed_file', '#title' => $this->t('Upload Cover
            Letter (PDF)'), '#required' => TRUE, '#upload_validators' => ['FileExtension' =>
            ['extensions' => 'pdf']]];

        $form['actions']['submit'] = ['#type' => 'submit', '#value' => $this->t('Apply Now')];

        return $form;
    }

    // Crochet natif execute lors du cycle POST avant soumission
    public function validateForm(array &$form, FormStateInterface $form_state) {
        $phone = $form_state->getValue('phone');
        // Validation stricte du telephone marocain via expression reguliere
        if (!preg_match('/^(06|07)[0-9]{8}$/ ', $phone)) {
            // Signalement de l'anomalie : bloque le cycle FormAPI et renvoie le formulaire
            // avec l'erreur
            $form_state->setErrorByName('phone', $this->t('The phone number must be 10 digits
                and start with 06 or 07.'));
        }
    }

    // Execution metier finale si l'etape validateForm s'est achevee avec succes.
    public function submitForm(array &$form, FormStateInterface $form_state) {
        // Extraction des donnees desormais validees et securisees contre les attaques HTML/
        // CSS
        $values = [
            'applicant_name' => $form_state->getValue('applicant_name'),
            'email' => $form_state->getValue('email'),
            'phone' => $form_state->getValue('phone'),
            'cv' => $form_state->getValue('cv'),
            'cover_letter' => $form_state->getValue('cover_letter'),
            'start_date' => $form_state->getValue('start_date'),
        ];

        // Delegation du poids du traitement metier au service JobBoardService instancie
        if ($this->jobBoardService->processApplication($values)) {
            // Message Flash signalant au navigateur visiteur le franc succes de l'operation
            $this->messenger()->addStatus($this->t('Application submitted!'));
        }
    }
}

```

```
}
}
```

4.2 Encapsulation via Bloc : JobApplicationBlock

L'instance de formulaire n'est pas routable en soi pour l'affichage public sans structure hôte. L'exposition du formulaire est effectuée par l'encapsulation de sa méthode de construction au sein d'un composant Bloc de Drupal, instancié via l'annotation `@Block`.

```
<?php
namespace Drupal\job_board\Plugin\Block;

use Drupal\Core\Block\BlockBase;
use Drupal\Core\Access\AccessResult;
use Drupal\Core\Session\AccountInterface;

/**
 * Annotation "Plugin": elle permet a la couche decouvreuse de Drupal
 * d'identifier automatiquement et de référencer cette entite PHP en tant que composant de
 * bloc.
 *
 * @Block(
 *   id = "job_application_block",
 *   admin_label = @Translation("Job Application Form Block"),
 *   category = @Translation("Job Board")
 * )
 */
class JobApplicationBlock extends BlockBase {

    // Methode centrale generatrice dictant le rendu du composant
    public function build() {
        // Utilisation du builder global FormBuilder pour resoudre et instancier formellement
        // la matrice (Render Array) retournee par le formulaire associe.
        return ['form' => \Drupal::formBuilder()->getForm('\Drupal\job_board\Form\
            JobApplicationForm')];
    }

    // Limitation des droits de consultation a certains niveaux
    protected function blockAccess(AccountInterface $account) {
        // Renvoie une permission si le visiteur dispose d'une autorisation stricte specifique
        return AccessResult::allowedIfHasPermission($account, 'submit job application');
    }
}
```

5 Contrôleurs, Dashboard et Vues

5.1 Affichage du Tableau de Bord : JobBoardAdminController

Les contrôleurs orchestrent le flux des données entre la couche de modèle et la couche de présentation. Le **JobBoardAdminController** interroge l'**EntityStorage** natif pour collecter la grappe d'entités candidates. Lors de l'itération des données, le contrôleur ne manipule pas la sémantique HTML. Il produit un *Render Array* typifié (`#theme => 'job_board_admin_dashboard'`), chargeant le moteur natif de résolution de thème d'appliquer le template Twig qui opère la transformation finale.

Des commentaires typographiques (`/** @var ... */`) informent l'analyseur sur les contrats d'interface, afin d'assurer l'inférence des types dynamiques retournés par la méthode générique `loadMultiple()`.

```
<?php
namespace Drupal\job_board\Controller;

use Drupal\Core\Controller\ControllerBase;
use Drupal\Core\Entity\EntityTypeManagerInterface;
```

```

use Symfony\Component\DependencyInjection\ContainerInterface;

class JobBoardAdminController extends ControllerBase {

    protected $entityTypeManager;

    // Pattern commun d'injection par dependance
    public function __construct(EntityTypeManagerInterface $entity_type_manager) { $this->
        entityTypeManager = $entity_type_manager; }
    public static function create(ContainerInterface $container) { return new static(
        $container->get('entity_type.manager')); }

    public function dashboard() {
        // Pointer vers l'arborescence SQL abritant nos entites metier
        $storage = $this->entityTypeManager->getStorage('job_application');
        // Requeter toutes les candidatures, contourner exceptionnellement
        // la file de restriction d'accès, et les trier de la plus recente a la plus ancienne
        $query = $storage->getQuery()->accessCheck(FALSE)->sort('id', 'DESC');
        // Execution et recuperation d'un tableau des Entity IDs primaires
        $sais = $query->execute();

        // Chargement lourd exhaustif des donnees via les IDs
        $applications = $storage->loadMultiple($sais);
        $rows = [];
        $file_storage = $this->entityTypeManager->getStorage('file');

        /** @var \Drupal\job_board\JobApplicationInterface $app */
        foreach ($applications as $app) {
            // Tentative par resolution de capturer les URLs de telechargement pour les
            // fichiers
            $cv_url = '#'; if ($app->getCvId()) { if ($cv = $file_storage->load($app->getCvId(
                ))) $cv_url = $cv->createFileUrl(FALSE); }
            $cl_url = '#'; if ($app->getCoverLetterId()) { if ($cl = $file_storage->load($app
                ->getCoverLetterId())) $cl_url = $cl->createFileUrl(FALSE); }

            // Construction iterative d'un tableau pre-mache et securise de variables
            // a déposer dans l'assise du moteur Twig associe
            $rows[] = [
                'id' => $app->id(),
                'name' => $app->getApplicantName(),
                'email' => $app->get('email')->value,
                'phone' => $app->getPhone(),
                'start_date' => $app->getStartDate(),
                'cv_url' => $cv_url,
                'cl_url' => $cl_url,
                'status' => $app->getStatus(),
            ];
        }

        // Renvoi du Render Array decanille explicitement vers
        // l'identifiant 'job_board_admin_dashboard' inscrit dans function job_board_theme()
        return [
            '#theme' => 'job_board_admin_dashboard',
            '#applications' => $rows,
        ];
    }
}

```

5.2 Traitement des Décisions RH : JobBoardActionController

Ce contrôleur expose les primitives nécessaires pour router les actions d'interface (Acceptation/Rejet). Bien que ces actions soient déclenchées par des requêtes HTTP GET (via de simples liens cliquables), le contrôleur implémente un mécanisme de redirection sécurisée une fois l'action accomplie.

Cela permet d'éviter l'exécution répétée de l'action (le problème du *Double Submit*) si l'administrateur rafraîchit la page (F5). La méthode **handleAction** modifie l'état de l'application via la couche de service, établit un message de notification flash en session, et retourne ultimement une classe **RedirectResponse**. Cela force le navigateur client à

revenir proprement sur le tableau de bord originel, purgeant par la même occasion la chaîne de paramètres (ex. `/accept/3`) de l'historique url actif du navigateur.

```
<?php
namespace Drupal\job_board\Controller;

use Drupal\Core\Controller\ControllerBase;
use Symfony\Component\DependencyInjection\ContainerInterface;
use Symfony\Component\HttpFoundation\RedirectResponse;
use Drupal\Core\Url;
use Drupal\job_board\Service\JobBoardService;

class JobBoardActionController extends ControllerBase {

    protected $jobBoardService;

    // Injection via Interface
    public function __construct(JobBoardService $job_board_service) {
        $this->jobBoardService = $job_board_service;
    }
    public static function create(ContainerInterface $container) {
        return new static($container->get('job_board.service'));
    }

    // Points de chute des routes dynamiques 'accept/{id}' et 'reject/{id}' définies dans
    // routing.yml.
    // L'identifiant dans l'URL ({id}) est récupéré en tant que paramètre d'appel formel par
    // Symfony.
    public function accept($id) { return $this->handleAction($id, 'application_accepted', '
    Accepted'); }
    public function reject($id) { return $this->handleAction($id, 'application_rejected', '
    Rejected'); }

    // Logique de délégation centralisée
    protected function handleAction($id, $mail_key, $action_name) {
        // Envoi effectif de la commande fonctionnelle à la couche de service
        $result = $this->jobBoardService->processAction($id, $mail_key, $action_name);

        // Analyse statique des résultats pour retour immédiat et de type booléen/null
        // avec construction de Feedback textuels via l'API Messenger de Drupal (bannières UI)
        if ($result === null) {
            $this->messenger()->addError($this->t('Application not found.'));
        } elseif ($result === true) {
            $this->messenger()->addStatus($this->t('Decision sent!'));
        } else {
            $this->messenger()->addError($this->t('Email failed to send.'));
        }

        // La classe \Drupal\Core\Url récupère formellement le chemin associé à l'identifiant
        // "job_board.admin_dashboard" et procède à l'instantiation
        // d'une RedirectResponse terminale, ce qui vide la chaîne GET de l'historique et
        // prévient le comportement inopiné au rafraîchissement (F5).
        return new RedirectResponse(Url::fromRoute('job_board.admin_dashboard')->toString());
    }
}
```

5.3 Gabarit Visuel : job-board-admin-dashboard.html.twig

Le *Render Array* du contrôleur d'affichage appelle explicitement la résolution d'un template Twig pour sérier ses variables sous un format HTML lisible. Ce fichier doit impérativement exister dans le répertoire **templates/** du module pour que le moteur thématique Drupal y puise sa structure.

```
{# Fichier : templates/job-board-admin-dashboard.html.twig #}
<div class="job-board-dashboard">
    <h2>{{ 'Job Applications'|t }}</h2>

    {% if applications is empty %}
        <p>{{ 'No applications have been submitted yet.'|t }}</p>
```

```

{% else %}
  <table class="job-applications-table">
    <thead>
      <tr>
        <th>{{ 'ID'|t }}</th>
        <th>{{ 'Name'|t }}</th>
        <th>{{ 'Email'|t }}</th>
        <th>{{ 'Phone'|t }}</th>
        <th>{{ 'Start Date'|t }}</th>
        <th>{{ 'Documents'|t }}</th>
        <th>{{ 'Actions'|t }}</th>
      </tr>
    </thead>
    <tbody>
      {# Boucle d'iteration sur le Render Array "applications" compile par le
        controleur #}
      {% for app in applications %}
        {# Classe CSS dynamique inferree depuis l'etat (pending, accepted, rejected
          ) formatee intelligemment via twig clean_class #}
        <tr class="status-{{ app.status|clean_class }}">
          <td>{{ app.id }}</td>
          <td>{{ app.name }}</td>
          <td><a href="mailto:{{ app.email }}">{{ app.email }}</a></td>
          <td>{{ app.phone }}</td>
          <td>{{ app.start_date|date('Y-m-d') }}</td>
          <td>
            {# Verification de la presence conditionnelle des pieces jointes
              #}
            {% if app.cv_url != '#' %}
              <a href="{{ app.cv_url }}" target="_blank">{{ 'View CV'|t }}</a><br>
            {% endif %}
            {% if app.cl_url != '#' %}
              <a href="{{ app.cl_url }}" target="_blank">{{ 'View Cover
                Letter'|t }}</a>
            {% endif %}
          </td>
          <td>
            {# Disparition de la visibilite des liens d'actions sitot la
              candidature debattue #}
            {% if app.status == 'pending' %}
              <a href="{{ path('job_board.action.accept', {'id': app.id}) }}"
                class="button accept-btn">{{ 'Accept'|t }}</a>
              <a href="{{ path('job_board.action.reject', {'id': app.id}) }}"
                class="button reject-btn">{{ 'Reject'|t }}</a>
            {% else %}
              <strong>{{ app.status|capitalize }}</strong>
            {% endif %}
          </td>
        </tr>
      {% endfor %}
    </tbody>
  </table>
{% endif %}
</div>

```

6 Modificateurs Système (Hooks)

6.1 Fichier `job_board.module`

Le module emploie les points d'ancrage natifs (Hooks) de l'API Drupal pour intégrer son code sémantique aux flux système.

Le `hook_theme` indexe le gabarit Twig créé plus haut afin qu'il soit détectable par le *Render Array*. Le `hook_mail` permet de définir les matrices contextuelles d'emails transigés en conjonction avec le composant **MailManager**.

```
<?php
```

```

function job_board_theme($existing, $type, $theme, $path) {
  // Declaration du hook_theme pointant vers le fichier job-board-admin-dashboard.html.twig
  return [
    'job_board_admin_dashboard' => ['variables' => ['applications' => []], 'template' => 'job-
      board-admin-dashboard'],
  ];
}

function job_board_mail($key, &$message, $params) {
  $application = $params['application'];

  // Remplissage dynamique des e-mails en fonction de la cle passee par le MailManager
  switch ($key) {
    case 'application_received':
      $message['subject'] = t('Thank you for your application, @name', ['@name' =>
        $application->getApplicantName()]);
      $message['body'][] = t('We have received your job application. Thank you!');
      break;

    case 'application_accepted':
      $message['subject'] = t('Application Status: Accepted');
      $message['body'][] = t('Congratulations, your application has been accepted!');
      break;

    case 'application_rejected':
      $message['subject'] = t('Application Status: Update');
      $message['body'][] = t('Unfortunately, we will not be moving forward with your
        application at this time.');
```

7 Fluidité de l'Interface d'Administration (JavaScript)

Lors de l'activation des balises primaires, la fonction anonyme attache un écouteur d'évènement (**click**). Lorsque cet évènement est déclenché, le script repère la cellule du tableau parente (**closest('td')**), y sélectionne tous les liens (les boutons d'action) et les masque (**display = 'none'**). Il génère ensuite et injecte un nouvel élément texte (*Processing...*) en l'attente du rechargement HTTP induit par le script côté serveur.

```

/**
 * Drupal behavior: hide Accept/Reject buttons after one is clicked.
 * Uses the Drupal 10 `once()` API (core/once).
 */
(function (Drupal, once) {
  'use strict';

  Drupal.behaviors.jobBoardActions = {
    attach: function (context) {
      // once() garantit que l'ecouteur n'est attache qu'une seule fois,
      // meme si le DOM est recharge partiellement via AJAX.
      once('job-board-actions', '.job-board-table td:last-child a', context)
        .forEach(function (link) {
          link.addEventListener('click', function () {
            // Identifier la cellule contenant les boutons
            var actionCell = link.closest('td');

            // Masquer TOUS les boutons presents (Accept & Reject)
            actionCell.querySelectorAll('a').forEach(function (a) {
              a.style.display = 'none';
            });

            // Creer dynamiquement l'element "Processing..."
            var label = document.createElement('em');
            label.textContent = 'Processing...';
            actionCell.appendChild(label); // L'insérer a la place des boutons
          });
        });
    });
  });

```

```
    }  
    };  
  })(Drupal, once);
```

8 Conclusion

Par l'utilisation judicieuse de l'API Entity quant à l'abstraction des données persistantes, de la Form API pour la validation, de l'isolation du routage avec les Contrôleurs, et de l'encapsulation de la logique métier dans des Services indépendants, le MVP du module "Job Board" démontre une conformité stricte vis-à-vis des meilleurs standards d'ingénierie imposés par l'écosystème Drupal moderne.